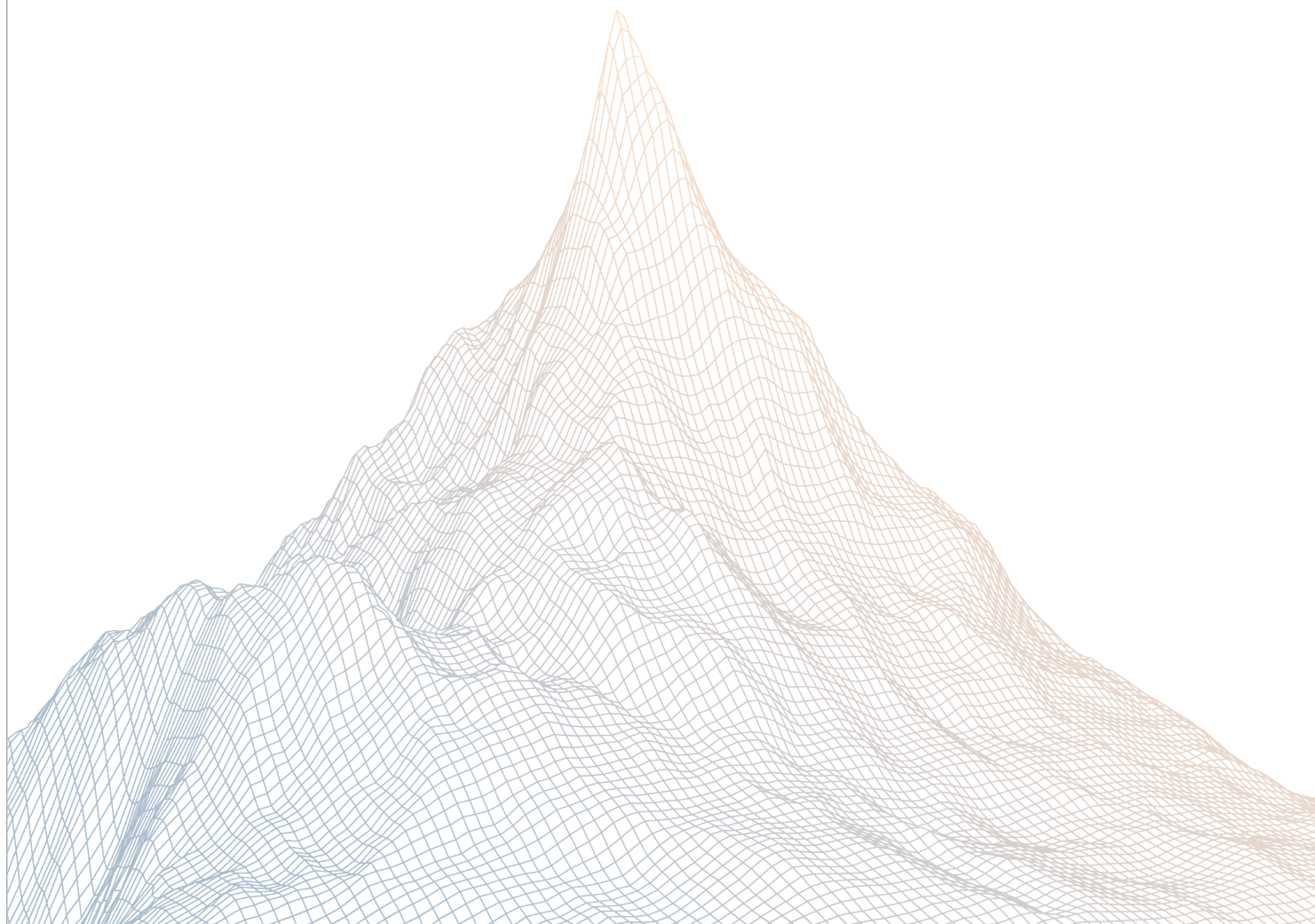


Meteora

Smart Contract Security Assessment

VERSION 1.1



Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	8
4.1	High Risk	9
4.2	Medium Risk	11
4.3	Low Risk	21
4.4	Informational	34

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	DLMM/PR-466
---------------	-------------

Repository	https://github.com/MeteoraAg/DLMM
-------------------	---

Commit Hash	97069565eca24b544d7809582f45acfcc5869384
--------------------	--

Files	Changes https://github.com/MeteoraAg/DLMM/pull/466
--------------	---

Target	DLMM/PR-479
---------------	-------------

Repository	https://github.com/MeteoraAg/DLMM
-------------------	---

Commit Hash	d5eb1d752b763733f08c6b75bf16cb062d61e0a7
--------------------	--

Files	Changes https://github.com/MeteoraAg/DLMM/pull/479
--------------	---

Target	Mitigation Review
Repository	https://github.com/MeteoraAg/DLMM
Commit Hash	c1aa2148389da8a607585eeea808b99f0fd56b3b
Files	Changes https://github.com/MeteoraAg/DLMM/pull/466 and https://github.com/MeteoraAg/DLMM/pull/479

2.3 Audit Timeline

February 2, 2026	Audit start
February 11, 2026	Audit end
February 27, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	5
Low Risk	6
Informational	7
Total Issues	19

3

Findings Summary

ID	Description	Status
H-1	swap2 exact-in under-accounts transfer fees allowing one to underpay for swaps	Resolved
M-1	Rounding in partial-filled limit order cancel allows over-withdrawal	Resolved
M-2	Incorrect bitmap state can cause swaps to skip bin with non-empty LO liquidity	Resolved
M-3	cancel_limit_order cannot distinguish valid lo_bin with bin id 0 from cancelled lo_bin	Resolved
M-4	Missing transfer fee handing in add_liquidity_by_weight2	Resolved
M-5	Pro-rata fee calculation allows late LO makers to steal previously accumulated LO fees	Acknowledged
L-1	Max bin limit can never be reached in place_limit_order	Acknowledged
L-2	Unnecessary is_ask check in LimitOrderBinData::is_empty	Resolved
L-3	Incorrect host fee calculation	Resolved
L-4	Limit order at active bin can be easily blocked by dust orders	Acknowledged
L-5	place_limit_order accepts LO with incorrect side for zero liquidity bins	Acknowledged
L-6	close_bin_array fails to ensure that fully filled LO are withdrawn	Resolved
I-1	Single-use intermediate support flags lead to unnecessary CU cost	Acknowledged
I-2	Redundant opposite feature checks across instructions	Resolved
I-3	Swap and Swap2 event emission order inconsistency	Resolved
I-4	Incorrect documentation for limit-order fee split	Resolved
I-5	Duplicate transfer_fee_excluded_amount_out > 0 check in swap2	Resolved
I-6	Incorrect and un-used comments	Resolved

ID	Description	Status
I-7	Fee split is incorrectly documented	Resolved

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] swap2 exact-in under-accounts transfer fees allowing one to underpay for swaps

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [swap2.rs#L414](#)
- [swap2.rs#L498](#)
- [swap2.rs#L538](#)

Description:

swap2 allows exact-in swaps based on the user-supplied amount_in and support Token2022, including transfer fee extension.

However, it transfers the swap input amount from user using swap_final_result.user_pay_amount, which is incorrect as it excludes transfer fees.

This will cause the pool to receive less input amount for the swap relative to the swap output, for pools with transfer fee tokens. This allows a malicious party to abuse it by swapping out more than expected, leading to a loss for the liquidity providers and LO users.

```
fn internal_handle_swap2<'c: 'info, 'info>(  
    ....  
    if exact_amount_out.is_none() {  
        // swap exact in  
        ctx.accounts.perform_swap(  
            swap_for_y,  
            fee_mode.fees_on_input,  
            swap_final_result.host_fee,  
            //@audit swap_final_result.user_pay_amount excludes transfer fee  
            swap_final_result.user_pay_amount,  
            swap_final_result.user_receive_amount,  
            transfer_hook_in_accounts,
```

```
        transfer_hook_out_accounts,  
    )?;  
} else {
```

Note that this also applies to the event emit in `build_swap_event()` and `build_swap_event2()`.

Recommendations:

1. This can be resolved by using the transfer fee included `amount_in` when transferring input amount from user for exact-in swaps.

```
if exact_amount_out.is_none() {  
    // swap exact in  
    ctx.accounts.perform_swap(  
        swap_for_y,  
        fee_mode.fees_on_input,  
        swap_final_result.host_fee,  
        swap_final_result.user_pay_amount,  
        amount_in,  
        swap_final_result.user_receive_amount,  
        transfer_hook_in_accounts,  
        transfer_hook_out_accounts,  
    );  
} else {
```

2. And update both `build_swap_event()` and `build_swap_event2()` to reflect the actual amount in.

Meteora: Resolved with [@7673dccc41 ...](#)

Zenith: Verified.

4.2 Medium Risk

A total of 5 medium risk findings were identified.

[M-1] Rounding in partial-filled limit order cancel allows over-withdrawal

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

Target

- [bin.rs#L906-L937](#)

Description

When a user cancels a limit order in `PartialFilled` status, the code splits their order size into fulfilled and unfilled portions using `calculate_shared_amounts` with `Rounding::Down` on the **unfilled** share:

```
LimitOrderStatus::PartialFilled => {
    let (unfilled_amount, fulfilled_amount) = calculate_shared_amounts(
        lo_bin.amount,
        self.processed_order_remaining_amount,
        self.total_processing_order_amount,
        Rounding::Down,
    )?;

    delta_processed_order_remaining_amount = unfilled_amount;
    delta_total_processing_order_amount = lo_bin.amount;

    (fulfilled_amount, unfilled_amount)
}
```

In `calculate_shared_amounts`, the first amount is computed with the given rounding (here, down), and the second is `amount - first_amount`. So `unfilled_amount` is rounded down and `fulfilled_amount = lo_bin.amount - unfilled_amount` is effectively **rounded up** (it can be one unit larger than the exact proportional share).

The user is then paid output tokens based on this `fulfilled_amount`:

```
// TODO check if round down is correct?
let swapped_amount =
  self.get_amount_in(fulfilled_amount, !lo_bin.is_ask(), Rounding::Down)?;
...
let (removed_amount_x, removed_amount_y) = if lo_bin.is_ask() {
  (unfilled_amount, swapped_amount)
} else {
  (swapped_amount, unfilled_amount)
};
```

Because `fulfilled_amount` can be one unit too high, the user is credited with slightly more “fulfilled” input than their true share, and receives the corresponding `swapped_amount` of output. Even with `Rounding::Down` in `get_amount_in`, the output can still correspond to that extra unit of input in edge cases, so the user can withdraw **more** of the output token (the fulfilled side) than they are owed.

Recommendations

We recommend splitting the order so that the **fulfilled** amount is rounded down (and unfilled rounded up), so the pool never credits the user with more fulfilled amount than the exact proportional share. For example, call `calculate_shared_amounts` with the ratio for the *fulfilled* part and `Rounding::Down` for that part, then set `unfilled_amount = lo_bin.amount - fulfilled_amount`. Keep `get_amount_in(fulfilled_amount, ..., Rounding::Down)` so the output token amount remains conservative from the pool’s perspective. This ensures users cannot withdraw more fulfilled output than they are owed.

Meteora: Resolved with [PR-487](#).

Zenith: Verified.

[M-2] Incorrect bitmap state can cause swaps to skip bin with non-empty LO liquidity

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [swap.rs#L263](#)
- [swap2.rs#L305](#)
- [bin.rs#L285-L293](#)

Description:

BinArray bitmaps are used to track whether the bin arrays have liquidity, so that it can be efficiently searched during swaps.

The bitmaps are updated whenever liquidity is added or removed, and not during swaps. This makes sense for MM liquidity since it gets swapped to the opposite side and remains in the bin array.

However, this is not correct for limit orders, as LO liquidity gets "removed" immediately after it is fulfilled during swaps. That means a LO-only bin array should be updated as "false" in the bitmap when all the LO within it are fulfilled (due to zero LO & MM liquidity). Instead, this issue causes the bitmap flag to remain as "true".

As the bitmap is not correctly updated after LO is fully consumed, it will cause the `LbPair::flip_bin_arrays()` to flip the bitmap incorrectly (setting it to `false` instead) when the next LO is placed in the same bin array. This will cause swaps to skip bins with actual LO liquidity.

Example Scenario

1. Suppose bin array X starts with no liquidity.
2. A user calls `place_limit_order` to place a LO in a bin array, which will flip the bitmap bit to `true` for bin array X.
3. After some time, the swap fully consumes the LO but the bitmap flag for bin array X stays `true` despite having zero liquidity after executing the LO.

NOTE: that this is the root cause, as the bitmap flag status is wrong.⁴ Now another user

calls `place_limit_order` to place a new LO in bin array X. That will obtains the "before" state using `get_zero_liquidity_flags()`, and determined that bin array X has `is_zero_liquidity() = true` (empty).⁵ Following that, `flip_bin_arrays()`, which obtain the "after" state, showing that bin array X has `is_zero_liquidity() = false` (non-empty) as the new LO has just been placed in it.

6. Since the before $\neq$ after, `flip_bin_arrays()` will flip the bit now to false, making the bitmap wrong, even though a new LO has been placed.
7. Due to the wrong bitmap, swaps will skip bin array X despite having LO liquidity.

Recommendations:

Ideally, this can be resolved by modifying swap to update the liquidity flag for the bin array when LO liquidity is depleted, though this could make swap less efficient.

Alternatively, this can be resolved by updating `Bin::is_zero_liquidity()` to check that `fulfilled_order_amount_x = 0 && fulfilled_order_amount_y = 0` when it is used by `BinArrayManager::get_zero_liquidity_flags()`. This will reflect the correct "before" state that can be used by `flip_bin_arrays()`. Though, this means that bin array with depleted LO liquidity will not be skipped, since the bitmap still reflects non-zero liquidity.

Meteora: Resolved with [@30cc532300 ...](#)

Zenith: Verified. Resolved by modifying swap/swap2 instructions to update the bin array liquidity flag when LO liquidity is depleted.

[M-3] `cancel_limit_order` cannot distinguish valid `lo_bin` with bin id 0 from cancelled `lo_bin`

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [limit_order.rs#L132-L142](#)
- [limit_order.rs#L144-L151](#)

Description:

`cancel_limit_order` will cancel the limit order for the specified bins and transfer the filled/unfilled amounts back to the owner.

For each specified bin, it will first look up `lo_bin` using `limit_order.get_limit_order_bin_data_mut_checked(bin_id)`.

And then `bin.apply_cancel_limit_order(lo_bin, &result)` will call `lo_bin.cancel_order()` and set all its fields including `bin` to zero.

However, this is problematic when the specified bins parameter contains bin 0, as `limit_order.get_limit_order_bin_data_mut_checked()` cannot distinguish between cancelled `lo_bin` from a valid `lo_bin` with bin id 0.

For example, if the LO has `lo_bin` for bin id `[-1, 0]`, cancelling `lo_bin` for `-1` will update it to `[0, 0]`. So when it tries to cancel the next valid `lo_bin` for `0`, it will incorrectly retrieve the cancelled `lo_bin` instead of the valid `lo_bin`.

```
pub fn get_limit_order_bin_data_mut_checked(
    &mut self,
    bin_id: i32,
) -> Result<&mut LimitOrderBinData> {
    for limit_order in self.limit_order_bins.iter_mut() {
        // @audit this will return cancelled lo_bin when searching for bin_id
        // = 0
        if limit_order.bin_id == bin_id {
            return Ok(limit_order);
        }
    }
}
```

```
Err(LBError::CannotFindLimitOrderByBinId.into())  
}
```

Note that `get_limit_order_bin_data_checked` has the same issue.

Recommendations:

Update both `get_limit_order_bin_data_mut_checked` and `get_limit_order_bin_data_checked` as follows,

```
pub fn get_limit_order_bin_data_mut_checked(  
    &mut self,  
    bin_id: i32,  
) -> Result<&mut LimitOrderBinData> {  
    for limit_order in self.limit_order_bins.iter_mut() {  
        if limit_order.bin_id == bin_id {  
            if limit_order.bin_id == bin_id && !limit_order.is_empty() {  
                return Ok(limit_order);  
            }  
        }  
    }  
    Err(LBError::CannotFindLimitOrderByBinId.into())  
}  
  
pub fn get_limit_order_bin_data_checked(&self, bin_id: i32) ->  
    Result<&LimitOrderBinData> {  
    for limit_order in self.limit_order_bins.iter() {  
        if limit_order.bin_id == bin_id {  
            if limit_order.bin_id == bin_id && !limit_order.is_empty() {  
                return Ok(limit_order);  
            }  
        }  
    }  
    Err(LBError::CannotFindLimitOrderByBinId.into())  
}
```

Meteora: Resolved with [@30d68a196b...](#)

Zenith: Verified. Resolved by not resetting limit order `bin_id` to zero and checking cancelled limit order is non-empty.

[M-4] Missing transfer fee handing in `add_liquidity_by_weight2`

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [add_liquidity_by_weight2.rs#L28](#)
- [add_liquidity_by_weight2.rs#L18](#)

Description:

`add_liquidity_by_weight2` allocates deposited tokens into bins based on user-specified weights to add liquidity.

However, it does not exclude for Token-2022 transfer fees when computing the effective deposited amounts, unlike `add_liquidity2` and `add_liquidity_by_strategy2`.

1. This could cause `add_liquidity_by_weight2` to deposit more funds than specified by the user, because `handle_deposit_by_amounts2()` will include transfer fee to the amounts in `amounts_in_bin`.
2. Another issue is that the `liquidity_parameter.validate(active_id)` performs validation on the pre-transfer fee amount `x` and amount `y`, which does not reflect the actual deposited amount. Though there is likely no impact as both amount `x` and amount `y` would still be > 0 after transfer fee and that would not change the outcome of the validation.

```
pub fn handle<'c: 'info, 'info>(  
    ctx: Context<_, '_, 'c, 'info, AddLiquidity2<'info>>,  
    liquidity_parameter: &LiquidityParameterByWeight,  
    remaining_accounts_info: RemainingAccountsInfo,  
) -> Result<()> {  
    let amounts_in_bin = {  
        let lb_pair = ctx.accounts.lb_pair.load()?;  
        let active_id = lb_pair.active_id;  
        let bin_step = lb_pair.bin_step;  
        liquidity_parameter.validate(active_id)?;  
        let (amount_x, amount_y)
```

```
= if liquidity_parameter.is_include_bin_id(active_id) {
    let transfer_hooks_account_len:
usize = remaining_accounts_info.total_length().into();
    if transfer_hooks_account_len > ctx.remaining_accounts.len() {
        return Err(LBError::InsufficientRemainingAccounts.into());
    }
    let bin_arrays_accounts = &mut
&ctx.remaining_accounts[transfer_hooks_account_len..];
    find_amount_in_active_bin(ctx.accounts.lb_pair.key(), active_id,
bin_arrays_accounts)?
    } else {
        (0, 0)
    };

    liquidity_parameter.to_amounts_into_bin(active_id, bin_step,
amount_x, amount_y)?
};
handle_deposit_by_amounts2(&ctx, &amounts_in_bin,
&remaining_accounts_info.slices)
}
```

Recommendations:

1. Update `add_liquidity_by_weight2` to handle transfer-fee using `calculate_transfer_fee_excluded_amount` within `liquidity_parameter.to_amounts_into_bin()`.
2. Consider updating `liquidity_parameter.validate(active_id)` to perform the `amount_x  $\neq$  0` and `amount_y  $\neq$  0` using transfer fee excluded amounts.

Meteora: Resolved with [@2503e37d34...](#)

Zenith: Verified.

[M-5] Pro-rata fee calculation allows late LO makers to steal previously accumulated LO fees

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [bin.rs#L989-L1038](#)

Description:

`calculate_limit_order_fee()` computes the LO fees owed to a limit order maker when cancelling a filled/partially-filled order.

However, the LO fees accumulate across multiple fill cycles and are never reset, but cancellation distributes fees using a simple pro-rata fee sharing, `limit_order_fee = limit_order_fee_side * user_fulfilled / total_fulfilled_order_amount`.

This allows a later participant to add a large fulfilled amount (e.g., via self-swaps) to dilute the share of existing makers and withdraw a disproportionate share of previously accumulated fees, effectively stealing fees from earlier makers who have not yet cancelled.

The fee theft is feasible when the fee rate is higher for the earlier LOs (e.g. due to variable fee), leading to a large amount of accumulated LO fees. That means late-comers can then steal from accumulated LO fees when the fee rate drops, as explained below.

Attack Scenario:

1. User A placed 100 X ask-side, fully filled in cycle 1. Swaps occurred during high volatility and accumulated `limit_order_fee_ask_side = 10`, `fulfilled_order_amount_x = 100`. User A has not cancelled yet.
2. Attacker places 1000 X ask-side order.
3. Attacker self-swaps $Y \rightarrow X$ to fill their order. Now `fulfilled_order_amount_x = 1100`, `limit_order_fee_ask_side = 15` (10 old + 5 new). Note that the fee is lower now due to lower volatility.
4. Attacker cancels: gets $15 * 1000/1100 \approx 13$ in fees (contributed only 5, stole ~8).
5. User A cancels: gets $2 * 100/100 = 2$ instead of their fair share of 10.

Note: The attacker's cost is ~50% of the self-swap trading fee (the protocol share). The stolen amount can far exceed this cost when accumulated fees are large relative to the existing fulfilled amounts.

Recommendations:

Modify the fee accounting such that each LO maker can only claim fees generated during the period their LO contributed to the swaps.

Meteora: Acknowledged.

4.3 Low Risk

A total of 6 low risk findings were identified.

[L-1] Max bin limit can never be reached in place_limit_order

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [place_limit_order.rs](#)
- [constants.rs#L17](#)

Description

The program allows up to **50 bins** per limit order (`MAX_BIN_PER_LIMIT_ORDER_USIZE`) and accepts **remaining accounts** for Token-2022 transfer hooks and for **bin array** PDAs. Solana legacy transactions are limited to **1232 bytes**. At least **one bin-array account is always required** (the bins must live in one or more bin arrays), so $X \geq 1$. With that minimum, a `place_limit_order` with the maximum 50 bins already exceeds 1232 bytes—the **50-bin cap is therefore infeasible for legacy transactions even without any transfer hook**. With a Token-2022 transfer hook the size grows further. Transactions that exceed the limit fail at submission or are dropped by the network.

Transaction size layout (legacy, 2 signers: user + limit-order PDA)

Part	Size (bytes)
Signatures	$1 + 2 \times 64 = 129$
Message header	3
Account list length	1
Account keys	$(12 + H + X) \times 32$
Recent blockhash	32
Instruction count	1
Program index	1
Instruction account count	1
Instruction account indices	$11 + H + X$
Instruction data length	1 or 2 (shortvec)
Instruction data	$39 + 12N$ (params) + RemainingAccountsInfo

- **N** = number of bins in `params.bins` (1–50).
- **H** = number of transfer-hook accounts in the single transfer-hook slice (TransferHookX or TransferHookY, depending on `is_ask_side`). At most one such slice is used; 0 if no hook.
- **X** = number of bin-array accounts (bins span one or more arrays; each array holds up to 70 bins). **X ≥ 1 always** (at least one bin array is required to place any limit order).

Instruction data:

- Discriminator: 8, `is_ask_side`: 1, padding: 16, `relative_bin` (Some): 9, bins: $4 + 12N$, so **params = $38 + 12N$** . With empty RemainingAccountsInfo the client may serialize only params → **$39 + 12N$** (or $42 + 12N$ / $43 + 12N$ if RemainingAccountsInfo is included; see discussion in audit).
- **RemainingAccountsInfo** (when present): 4 (vec length) + per slice 2 bytes (`accounts_type` + length). One transfer-hook slice adds 6 bytes.

For **N = 50**, params alone are 639 bytes; with RemainingAccountsInfo (one slice) → 644 bytes, so instruction data length uses **2 bytes** (shortvec).

Total size (simplified):

- Base (fixed accounts, no remaining): $129 + 3 + 1 + 12 \times 32 + 32 + 1 + 1 + 1 + 11 + 2 + (39 + 12N) \rightarrow$ with $N=50$ and 2-byte data length: $\approx 1219 + 33 \times (H + X)$.
- Each **additional account** (hook or bin array) adds **33 bytes** (32 in account list + 1 in instruction indices).

So:

- **Size $\approx 1219 + 33(H + X)$** for 50 bins (with 2-byte data length and $39+12N$ params).
- With **50 bins** in 1 bin array ($X = 1$) and **no** transfer hook: ≈ 1252 bytes $\rightarrow$ already **over 1232**.
- With **50 bins** ($X \geq 1$) and **H = 2** transfer-hook accounts (single hook slice): $\approx 1252 + 66 = 1318$ bytes $\rightarrow$ well over the limit.

Hence, **the advertised maximum of 50 bins is unreachable in a single legacy transaction**: at least one bin array is always required, and 50 bins in one bin array ($X = 1$) with no transfer hook already yields ≈ 1252 bytes, above 1232. With a transfer hook, the size is larger still.

Recommendations

We recommend reducing the `MAX_BIN_PER_LIMIT_ORDER_USIZE` variable to 48 so that in the case of no transfer hook and all the bins being within one array, the max can be reached.

Meteora: Acknowledged.

[L-2] Unnecessary is_ask check in LimitOrderBinData::is_empty

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [limit_order.rs#L21-32](#)

Description:

LimitOrderBinData::is_empty is used to determine whether a limit-order bin slot has no order (e.g. in LimitOrder::is_empty and close_limit_order_if_empty). The implementation is:

```
pub fn is_empty(&self) → bool {  
    self.amount == 0 && self.age == 0 && self.is_ask == 0  
}
```

Whether a bin slot is empty should depend only on there being no order amount. The amount field is the only one that directly represents “has an order or not”; when amount = 0, the slot is effectively empty. Including is_ask = 0 in the check is irrelevant for that meaning: is_ask only indicates side (ask vs bid) and does not indicate presence of an order. As a result:

- If a slot ever has amount = 0 and age = 0 but is_ask ≠ 0 (e.g. leftover from a bug, partial init, or future code path), is_empty would incorrectly return false and the limit order could be treated as non-empty when it has no size.
- The condition is redundant when cancel_order correctly zeroes all fields, but it tightens the definition of “empty” in a way that can cause inconsistent behavior if only amount (and age) are cleared elsewhere.

Recommendations:

We recommend changing is_empty to check only the fields that define an empty slot, e.g. self.amount == 0 (and optionally self.age == 0 for consistency with cancel_order). Remove the self.is_ask == 0 check so that “empty” is determined solely by whether there is an order amount.

Meteora: Resolved with [PR-476](#).

Zenith: Verified.

[L-3] Incorrect host fee calculation

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: High

Target

- [fee_calculator.rs#L89-L145](#)

Description

The inline documentation for `split_fees` states that the host fee (when referral is present) is:

```
host_fee (if have) = 20% * protocol_fee_on_mm_fee + 20% *  
protocol_fee_on_limit_order_fee
```

The implementation, however, computes the limit-order portion of the host fee differently. For the MM part it correctly takes 20% of `protocol_fee_on_mm_fee`. For the limit-order part it first multiplies `protocol_fee_on_limit_order_fee` by `protocol_fee_bps / BASIS_POINT_MAX`, then takes 20% of that intermediate value. So the actual formula is:

```
host_fee = 20% * protocol_fee_on_mm_fee + 20% * (protocol_fee_bps / 10000) *  
protocol_fee_on_limit_order_fee
```

This can be seen in the code here:

```
let host_fee = if has_referral {  
    let (host_fee_based_on_mm_amount, _) = calculate_shared_amounts(  
        protocol_fee_on_mm_fee,  
        HOST_FEE_BPS.into(),  
        BASIS_POINT_MAX.into(),  
        Rounding::Down,  
    );  
  
    // round down for host fee  
    let (virtual_protocol_fee_on_limit_order_fee, _)  
    = calculate_shared_amounts(  
        protocol_fee_on_limit_order_fee,  
        protocol_fee_bps.into(),  
    );  
}
```

```
        BASIS_POINT_MAX.into(),
        Rounding::Down,
    )?;

    let (host_fee_based_on_limit_order_amount, _)
    = calculate_shared_amounts(
        virtual_protocol_fee_on_limit_order_fee,
        HOST_FEE_BPS.into(),
        BASIS_POINT_MAX.into(),
        Rounding::Down,
    )?;

    host_fee_based_on_mm_amount.safe_add(host_fee_based_on_limit_order_amount)?
} else {
    0
};

let protocol_fee = protocol_fee_on_mm_fee
    .safe_add(protocol_fee_on_limit_order_fee)?
    .safe_sub(host_fee)?;
```

Recommendations

We recommend deciding whether the intended host fee on limit-order protocol fee is (A) 20% of full `protocol_fee_on_limit_order_fee` (comment's formula), or (B) 20% of $(\text{protocol_fee_bps}/10000) * \text{protocol_fee_on_limit_order_fee}$ (current code).

If (A), change the implementation so the limit-order host fee is $20\% * \text{protocol_fee_on_limit_order_fee}$ (remove the intermediate `virtual_protocol_fee_on_limit_order_fee` step for host fee) and ensure `protocol_fee` is still computed so that all fee components sum correctly.

If (B), update the comment (and any external docs) to state: `host_fee (if have) = 20% * protocol_fee_on_mm_fee + 20% * (protocol_fee_bps/10000) * protocol_fee_on_limit_order_fee`.

Meteora: Resolved with [PR-476](#).

Zenith: Verified.

[L-4] Limit order at active bin can be easily blocked by dust orders

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [bin.rs#L858-L873](#)

Description:

`place_limit_order()` only allows placing of limit order on the same side as existing LOs.

When the bin does not have any existing LO, users are allowed to place on either Ask or Bid side.

However, this allows one to easily block LO at the active bin by creating dust LO, which will deny users from placing LO, until the dust LO is consumed.

```
pub fn place_limit_order(
    &mut self,
    amount: u64,
    bin_id: i32,
    user_ask_side: bool,
    limit_order_bin_data: &mut LimitOrderBinData,
) → Result<()> {
    let pending_limit_order_amount = self.get_pending_limit_order_amount()?;
    let bin_ask_side = self.is_limit_order_ask_side();
    if pending_limit_order_amount > 0 {
        // require both limit order and bin limit order ask side must be same
        side
        require!(
            user_ask_side == bin_ask_side,
            LLError::InvalidPlaceLimitOrderParameters,
        );
        self.open_order_amount = self.open_order_amount.safe_add(amount)?;
    } else {
        // if not the same ask side and total_processing_order_amount is
        non-zero, we need to mark limit order is full-filled by increase
        order_age
        if user_ask_side != bin_ask_side &&
```

```
self.total_processing_order_amount > 0 {  
    self.total_processing_order_amount = 0;  
    self.order_age = self.order_age.safe_add(1)?;  
}  
self.set_limit_order_side(user_ask_side);  
self.open_order_amount = amount;  
}  
  
limit_order_bin_data.initialize(amount, bin_id, self.order_age,  
user_ask_side);  
Ok()  
}
```

Recommendations:

Consider adding a minimum amount for LO at the active bin.

Meteora: Acknowledged.

[L-5] `place_limit_order` accepts LO with incorrect side for zero liquidity bins

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [place_limit_order.rs#L125-L137](#)

Description:

`place_limit_order` uses `lb_pair.active_id` as the “price anchor” to validate Ask LO $\text{bin_id} \geq \text{active_id}$ and Bid LO $\text{bin_id} \leq \text{active_id}$.

However, it does not verify that the bin at `active_id` has any liquidity. Because once `active_id` is sitting on zero-liquidity range, the ask/bid side check becomes meaningless as one can traverse across the bins and place LO on any arbitrary side.

An attacker can abuse this by placing "invalid" LO in zero liquidity bins (e.g. Ask LO $\text{bin_id} < \text{active_id}$) and block users from placing legitimate LO in those bins.

```
if self.is_ask_side {
    // min id must be greater than or equal threshold_bin_id
    require!(
        min_id ≥ threshold_bin_id,
        LLError::InvalidPlaceLimitOrderParameters
    );
} else {
    // max id must be smaller than or equal threshold_bin_id
    require!(
        max_id ≤ threshold_bin_id,
        LLError::InvalidPlaceLimitOrderParameters
    );
}
```

Recommendations:

Consider updating `place_limit_order` to verify that the `active_id` bin has non-zero liquidity.

Meteora: We ACK, since the issue exist even before Limit order introduced, attacker could do this even with market making liquidity.

[L-6] `close_bin_array` fails to ensure that fully filled LO are withdrawn

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [close_bin_array.rs#L39](#)
- [bin.rs#L285-L293](#)

Description:

`close_bin_array` is used to close out-of-range bin array accounts that has zero liquidity within it.

However, it uses `bin::is_zero_liquidity()`, which only determines that there are no unfulfilled limit orders. That is problematic as there could still be limit orders that are fully fulfilled and not yet withdrawn. Closing such bin array accounts is not correct.

```
impl Bin {
    pub fn is_zero_liquidity(&self, is_support_limit_order: bool) → bool {
        if is_support_limit_order {
            self.liquidity_supply == 0
            && self.open_order_amount == 0
            && self.processed_order_remaining_amount == 0
        } else {
            self.liquidity_supply == 0
        }
    }
}
```

Recommendations:

This can be resolved by updating `Bin::is_zero_liquidity()` to check `fulfilled_order_amount_x == 0 && fulfilled_order_amount_y == 0` and ensure that all fulfilled LO have been withdrawn.

Meteora: Resolved with [@d5eb1d752b...](#)

Zenith: Verified. Resolved by ensuring all the bins in the to-be-closed bin array are empty

(zerorized to default values).

4.4 Informational

A total of 7 informational findings were identified.

[I-1] Single-use intermediate support flags lead to unnecessary CU cost

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- [update_fees_and_reward2.rs#L50-L73](#)
- [claim_fee2.rs#L134-L157](#)
- [add_liquidity2.rs#L143-L164](#)
- [remove_liquidity2.rs#L152-L173](#)

Description

In multiple functions, the result of `lb_pair.support_liquidity_mining()`? is stored in an intermediate variable that is only referenced once.

```
let is_support_liquidity_mining = lb_pair.support_liquidity_mining()?;
...
if is_support_liquidity_mining {
    bin_array_manager.update_rewards(&mut lb_pair, current_timestamp)?;
}
```

Recommendations

For single-use cases, inline the check at the use site:

```
if lb_pair.support_liquidity_mining()? {
    ...
}
```

Meteora: Acknowledged.

[I-2] Redundant opposite feature checks across instructions

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [add_liquidity.rs#L244-L251](#)
- [remove_liquidity.rs#L87-L102](#)
- [remove_all_liquidity.rs#L40-L57](#)
- [add_liquidity_by_weight_one_side.rs#L246-L253](#)
- [add_liquidity2.rs#L143-L145](#)
- [remove_liquidity2.rs#L152-L154](#)
- [add_liquidity_single_side_precise2.rs#L180-L186](#)
- [rebalance_wrapper.rs#L379-L387](#)

Description

Multiple instructions call both:

- `lb_pair.support_liquidity_mining()`?
- `lb_pair.support_limit_order()`?

even though these two feature checks are strict opposites by construction:

```
pub fn support_liquidity_mining(&self) → Result<bool> {
    Ok(self.pair_type == PairType::Permissionless || self.pair_type
    == PairType::Permission)
}

pub fn support_limit_order(&self) → Result<bool> {
    Ok(!self.support_liquidity_mining())?
}
```

Representative redundant usage pattern:

```
let is_support_limit_order = lb_pair.support_limit_order()?;
...
if lb_pair.support_liquidity_mining()? {
```

```
bin_array_manager.update_rewards(&mut lb_pair,  
current_timestamp.safe_cast())?);  
}
```

This leads to unnecessary complexity and CU cost.

Recommendations

We recommend evaluating one capability flag once per function and deriving the opposite via logical negation.

Meteora: Resolved with [PR-476](#).

Zenith: Verified.

[I-3] Swap and Swap2 event emission order inconsistency

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [swap.rs#L324-L345](#)
- [swap2.rs#L399-L418](#)
- [swap2.rs#L431-L439](#)

Description

The legacy SPL swap path and the Token-2022 swap path use different event emission timing:

- swap emits events only after token transfers complete.

```
// swap.rs: transfer first, emit events after successful transfer
ctx.accounts.perform_swap(
    swap_for_y,
    fee_mode.fees_on_input,
    swap_final_result.host_fee,
    swap_final_result.user_pay_amount,
    swap_final_result.user_receive_amount,
)?;

emit_cpi!(swap_final_result.build_swap_event(&params)?);
emit_cpi!(swap_final_result.build_swap_event2(&params, &fee_mode)?);
```

- swap2 emits events before token transfers are executed.

```
// swap2.rs: emit events first, transfer later
emit_cpi!(swap_final_result.build_swap_event(&params)?);
emit_cpi!(swap_final_result.build_swap_event2(&params, &fee_mode)?);

if exact_amount_out.is_none() {
    ctx.accounts.perform_swap(
        swap_for_y,
        fee_mode.fees_on_input,
```

```
        swap_final_result.host_fee,  
        swap_final_result.user_pay_amount,  
        swap_final_result.user_receive_amount,  
        transfer_hook_in_accounts,  
        transfer_hook_out_accounts,  
    )?;  
} else {  
    ctx.accounts.perform_swap(  
        swap_for_y,  
        fee_mode.fees_on_input,  
        swap_final_result.host_fee,  
        transfer_fee_included_amount_in,  
        swap_final_result.user_receive_amount,  
        transfer_hook_in_accounts,  
        transfer_hook_out_accounts,  
    )?;  
}
```

On Solana, logs can still be present for failed transactions. Therefore, if a swap2 transfer fails after events are emitted (for example due to transfer-hook/account mismatch or token program transfer failure), off-chain consumers that parse logs without checking transaction success may ingest a swap event that did not finalize on-chain state.

Recommendations

We recommend making event semantics consistent across both instructions.

Meteora: Resolved with [PR-476](#).

Zenith: Verified.

[I-4] Incorrect documentation for limit-order fee split

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [fee_calculator.rs#L87-L113](#)

Description

The inline documentation for `split_fees` states that `total_limit_order_fee` is split into `limit_order_fee` and `protocol_fee_on_limit_order_fee`:

```
total_limit_order_fee = limit_order_fee + protocol_fee_on_limit_order_fee = x%  
    * total_limit_order_fee + (100-x)% * total_limit_order_fee
```

The implementation does not perform this split for the limit-order side. The code assigns the entire `total_limit_order_fee` to `protocol_fee_on_limit_order_fee`:

```
// we take all limit order fee  
let protocol_fee_on_limit_order_fee = total_limit_order_fee;
```

So in practice `total_limit_order_fee` is equal to `protocol_fee_on_limit_order_fee` and there is no separate `limit_order_fee` portion (i.e. the effective split is 0% / 100%, not x% / (100-x)%). The comment is inconsistent with the implementation and can mislead readers about how limit-order fees are distributed.

Two further comments are wrong for the same reason:

- The comment `limit_order_fee = x% * total_limit_order_fee` (line 94) is wrong because `limit_order_fee` does not really exist in the implementation.
- The comment `protocol_fee + host_fee = 20% * total_mm_fee + (100-x)% * total_limit_order_fee` (line 95) is wrong: it should not be `(100-x)% * total_limit_order_fee` but just `total_limit_order_fee`.

Recommendations

We recommend updating the comments at lines 87, 93—95 to state that the protocol takes all of the limit-order fee. Alternatively, if a non-zero `limit_order_fee` share is intended, the implementation should be changed to split `total_limit_order_fee` via `calculate_shared_amounts` (or equivalent) in line with the MM fee handling above.

Meteora: Resolved with [PR-476](#).

Zenith: Verified.

[I-5] Duplicate transfer_fee_excluded_amount_out > 0 check in swap2

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [swap2.rs#L402-L406](#)
- [swap2.rs#L751-L755](#)

Description:

Both process_swap() and internal_handle_swap2() performs the same check to ensure non-zero output amount.

```
// When total_amount_out = 0, the in_amount for swap couldn't get swap out 1 amount
require!(
    transfer_fee_excluded_amount_out > 0,
    LbError::InsufficientOutAmount
);
```

One of them can be removed to save CU.

Recommendations:

Consider removing the redundant check in internal_handle_swap2().

Meteora: Resolved with [@d6830f8893...](#)

Zenith: Verified.

[I-6] Incorrect and un-used comments

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [bin.rs#L198](#)
- [bin.rs#L762](#)
- [lb_pair.rs#L73](#)
- [preset_parameters2.rs#L84-L99](#)

Description:

1. Consider correcting spelling error in the comment in [bin.rs#L198](#).

```
} else {  
    // otherwise just except it is dust in lb_pair  
    // otherwise just accept it is dust in lb_pair  
}
```

2. Consider correcting the comment in [bin.rs#L762](#).

```
/// Get maximum token amount needed to deposit into bin, in order to  
withdraw out all the opposite token from the bin.  
/// Get token amount needed to deposit into bin, in order to withdraw out  
the opposite token from the bin.  
/// X → Y: reserve_y / bin_price  
/// Y → X: reserve_x * bin_price  
pub fn get_amount_in(  

```

3. Consider updating the comment to be relevant in [lb_pair.rs#L73](#).

```
/// Collect fee mode. Unlike DAMMv2, we have three modes here instead of  
flipping token X and Y position due to liquidity strategy construction  
is based on quote token (Y)  
/// Collect fee mode.
```

4. Consider removing the un-used comment in preset_parameters2.rs#L84-L99.

```
// impl FunctionType {  
//     pub fn support_liquidity_mining(&self,  
//                                     reward_infos: &[RewardInfo; 2]) → bool {  
//         match self {  
//             FunctionType::Undetermined ⇒ {  
//                 // check whether reward infos was initialized  
//                 reward_infos.iter().any(|info| info.initialized())  
//             }  
//             FunctionType::LiquidityMining ⇒ true,  
//             FunctionType::LimitOrder ⇒ false,  
//         }  
//     }  
  
//     pub fn support_limit_order(&self,  
//                               reward_infos: &[RewardInfo; 2]) → bool {  
//         !self.support_liquidity_mining(reward_infos)  
//     }  
// }
```

Meteora: Resolved with [@7021c121aa...](#)

Zenith: Verified.

[I-7] Fee split is incorrectly documented

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/lb_clmm/src/fee_calculator.rs#L88-L96](#)
- [programs/lb_clmm/src/fee_calculator.rs#L106-L112](#)

Description:

programs/lb_clmm/src/fee_calculator.rs contains `split_fees`, which splits swap fees into market-maker fee, protocol fee, limit-order fee, and host fee. The actual split for the MM portion is implemented by using `protocol_fee_bps` in `calculate_shared_amounts`.

```
// programs/lb_clmm/src/fee_calculator.rs#L106-L112
let (protocol_fee_on_mm_fee, mm_fee) = calculate_shared_amounts(
    total_mm_fee,
    protocol_fee_bps.into(),
    BASIS_POINT_MAX.into(),
    Rounding::Down,
)?;
```

The comments above this logic document a fixed 80% / 20% MM split. This is inconsistent with the function behavior, because the code uses `protocol_fee_bps` and therefore supports different shares per pool.

```
// programs/lb_clmm/src/fee_calculator.rs#L88-L96
// total_mm_fee = mm_fee + protocol_fee_on_mm_fee = 80% * total_mm_fee + 20%
//   * total_mm_fee
// ...
// mm_fee = 80% * total_mm_fee
// ...
// protocol_fee + host_fee = 20% * total_mm_fee + (100-x)% *
//   total_limit_order_fee
```

Recommendations:

We recommend updating the `split_fees` comments in `programs/lb_clmm/src/fee_calculator.rs` to describe the MM split as `protocol_fee_bps / 10_000` and the remainder, instead of fixed 80% / 20% text. This keeps the documentation consistent with the actual parameter-driven logic.

```
// Suggested comment style in split_fees:  
// total_mm_fee = mm_fee + protocol_fee_on_mm_fee  
// protocol_fee_on_mm_fee = total_mm_fee * protocol_fee_bps / 10_000  
// mm_fee = total_mm_fee - protocol_fee_on_mm_fee
```

Meteora: Resolved with [PR-496](#).

Zenith: Verified.